

# **CS 635, Fall 2025**

## **Module 2 Assignment**

**Prepared by:**

Leo Paredes (820558278), Brandon Reynolds (821924175)

E-mail: [lparedes7353@sdsu.edu](mailto:lparedes7353@sdsu.edu) , [bareynolds@sdsu.edu](mailto:bareynolds@sdsu.edu)

San Diego State University

**Design:**

Our Adaptive Library Management System (ALMS) software design is architected using OOP principles to create a flexible and maintainable solution. A class hierarchy has been developed for primary entities such as Book, User, and Transaction that work well with inheritance for shared attributes and behaviors from BaseEntity. We also used encapsulation to help keep critical data fields safe such as user profiles, book availability, and transaction history. These data fields are still able to be accessed and modified exclusively through well-defined methods. We also utilized abstract classes and interfaces such as the Searchable interface for consistent behaviors across multiple classes in the repo. Lastly, we also were able to implement polymorphism through dynamic transaction processing through classes like BorrowTransaction and ReturnTransaction.

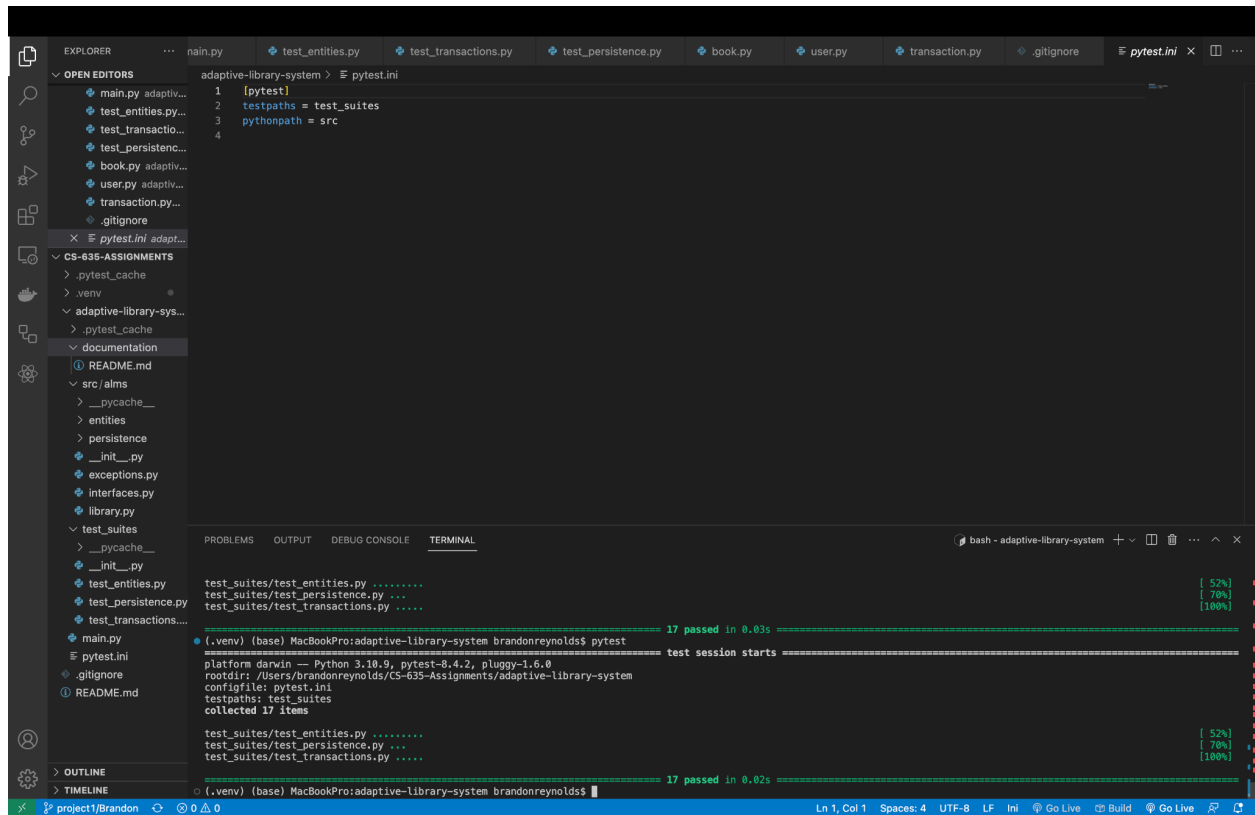
**Implementation:**

The implementation follows the design pretty closely. Transactions are handled flexibly thanks to polymorphism, and we added exception handling to deal with errors like borrowing unavailable books or invalid user input. For persistence, we created the class FileManager so data can be saved and loaded easily. We tried to keep everything modular as well in order to make it easy to read, test, and expand later if needed.

**Testing:**

For testing, we used pytest to introduce a testing suite for covering different aspects of the src code in the repo. We did this to test correctness and robustness of the system by having our comprehensive tests cover basic and advanced functionality. Examples of the scenarios tested were: successful and unsuccessful book transactions, user interactions, and edge cases involving invalid inputs. Our modular design allowed for targeted unit tests using frameworks like pytest. We also introduced [main.py](#) that presented the basic functionality of our system.

## Test Results:



```

1 [pytest]
2 testpaths = test_suites
3 pythonpath = src
4

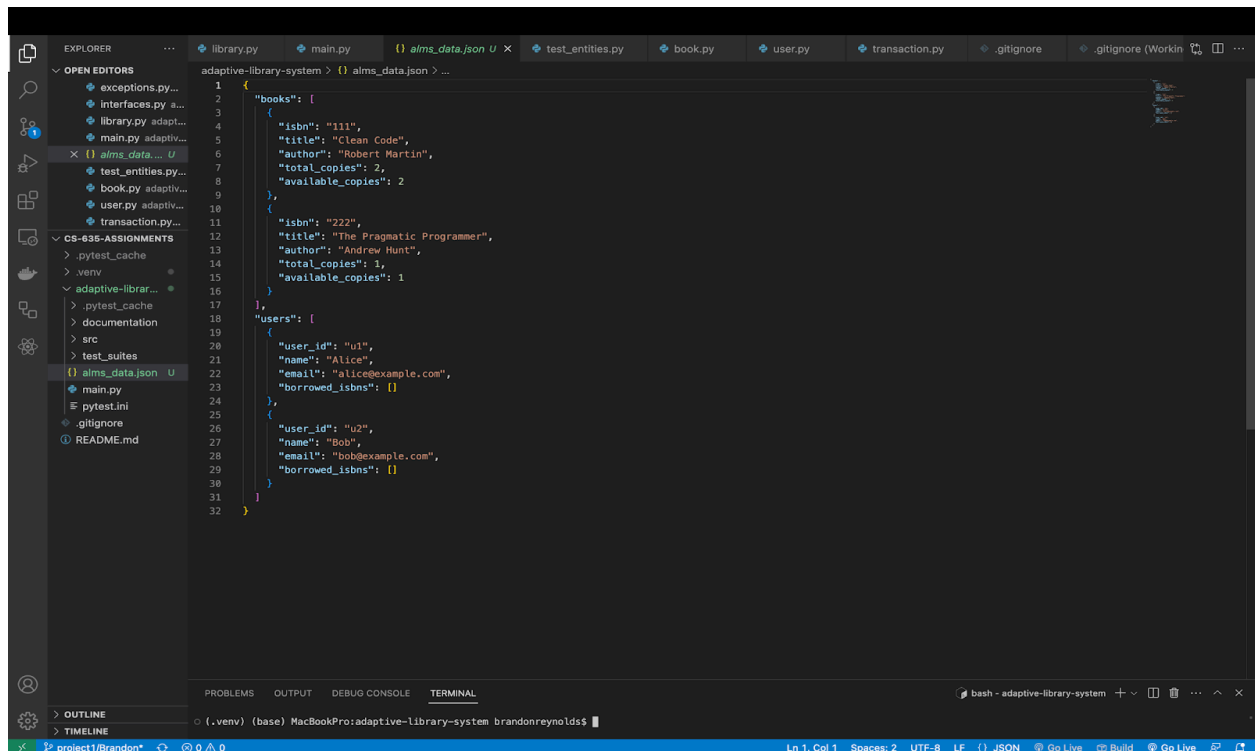
platform darwin -- Python 3.10.9, pytest-8.4.2, pluggy-1.6.0
rootdir: /Users/brandonreynolds/CS-635-Assignments/adaptive-library-system
configfile: pytest.ini
testpaths: test_suites
collected 17 items

test_suites/test_entities.py ..... [ 52%]
test_suites/test_persistence.py ... [ 70%]
test_suites/test_transactions.py ..... [100%]

===== 17 passed in 0.03s =====
test session starts
platform darwin -- Python 3.10.9, pytest-8.4.2, pluggy-1.6.0
rootdir: /Users/brandonreynolds/CS-635-Assignments/adaptive-library-system
configfile: pytest.ini
testpaths: test_suites
collected 17 items

test_suites/test_entities.py ..... [ 52%]
test_suites/test_persistence.py ... [ 70%]
test_suites/test_transactions.py ..... [100%]

===== 17 passed in 0.02s =====
  
```



```

1 {
2   "books": [
3     {
4       "isbn": "111",
5       "title": "Clean Code",
6       "author": "Robert Martin",
7       "total_copies": 2,
8       "available_copies": 2
9     },
10    {
11      "isbn": "222",
12      "title": "The Pragmatic Programmer",
13      "author": "Andrew Hunt",
14      "total_copies": 1,
15      "available_copies": 1
16    }
17  ],
18  "users": [
19    {
20      "user_id": "u1",
21      "name": "Alice",
22      "email": "alice@example.com",
23      "borrowed_isbns": []
24    },
25    {
26      "user_id": "u2",
27      "name": "Bob",
28      "email": "bob@example.com",
29      "borrowed_isbns": []
30    }
31  ]
32 }
  
```

The screenshot shows a VS Code editor with the following components:

- EXPLORER (Left Panel):** Displays the file structure of the 'adaptive-library-system' project. The 'test\_suites' directory is expanded, showing files like 'test\_entities.py', 'test\_persistence.py', and 'test\_transactions.py'. 'test\_persistence.py' is selected.
- EDITOR (Main Panel):** Shows the code for 'test\_persistence.py'. The code includes:
 

```

22 # Check for adding entries and searching for books and users not in library
23 def test_not_matching_book_and_user(tmp_path):
24     file_path = tmp_path / "test_alms_2.json"
25
26     lib = Library()
27     lib.add_book(Book("111", "Clean Code", "Robert Martin", total_copies=2))
28     lib.add_user(User("u1", "Alice", "alice@example.com"))
29
30     fm = FileManager(str(file_path))
31     fm.save(lib)
32
33     loaded = fm.load(Library)
34
35     # Test that these books and users don't exist in the file manager
36     book = loaded.get_book("111")
37     user = loaded.get_user("u1")
38
39     assert book.get_title() != "Intro to Programming"
40     assert user.get_name() != "Bob"
41
42 # Check for no files
43 def test_load_nonexistent_file(tmp_path):
44     file_path = tmp_path / "test_alms_3.json"
45     fm = FileManager(str(file_path))
46     lib = fm.load(Library)
47     assert isinstance(lib, Library)
      
```
- TERMINAL (Bottom Panel):** Shows the output of running 'python main.py'. The output is:
 

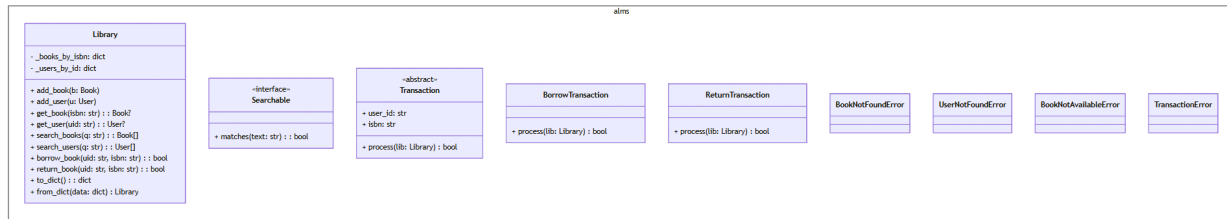
```

(base) brandonreynolds@MacBookPro adaptive-library-system % python main.py
== Books ==
111 Clean Code Robert Martin 2
222 The Pragmatic Programmer Andrew Hunt 1

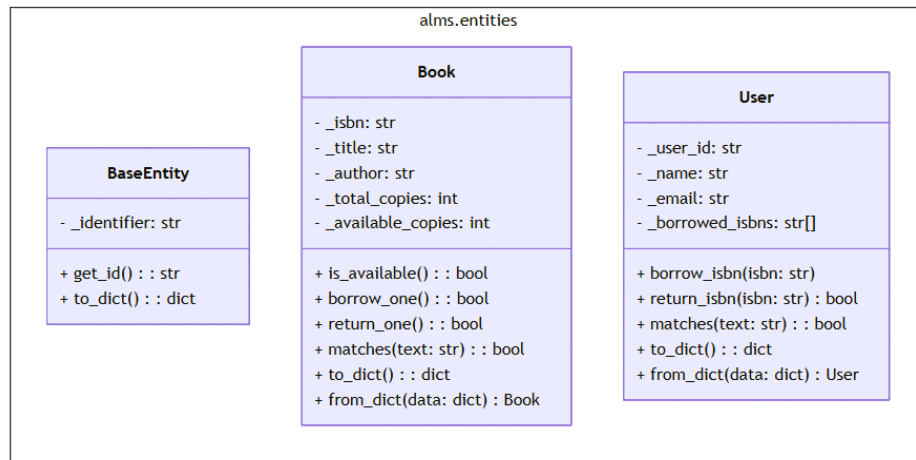
Borrow '111' for user 'u1'
Available copies for 111: 1

Return '111' for user 'u1'
Available copies for 111: 2
(base) brandonreynolds@MacBookPro adaptive-library-system %
      
```

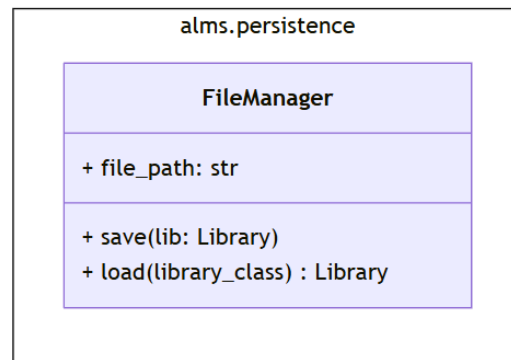
## UML Diagram:



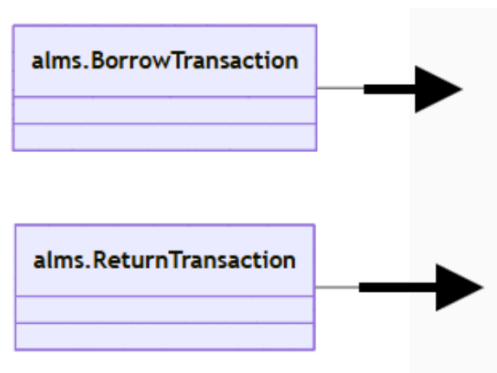
The diagram above is the core of the Adaptive Library Management System (ALMS). The ALMS package contains the library class that manages the collection of Book and User objects. It provides methods for adding entities (book, user, transaction). It contains the searchable interface for both Books and Users for flexible keyword search. Transaction is an abstract class that can define any transaction with its method. BorrowTransaction and ReturnTransaction extend Transaction through polymorphism and inheritance. Exceptions handle invalid operations like BookNotFoundError, UserNotFoundError, BookNotAvailableError, and TransactionError.



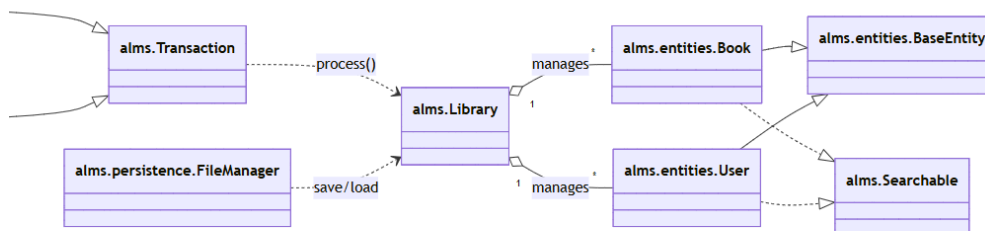
The diagram above is for the ALMS entities. The class BaseEntity provides common attributes and methods that both objects Book and User inherit. Book contains encapsulated behaviors to restrict access to data fields. It implements Searchable so books can be found through title, author, or ISBN. User also contains encapsulated behavior to restrict access to data fields and implements Searchable to allow lookup by ID, name, or email.



The diagram above Persistence is handled by the FileManager class. This allows the ALMS to save and retrieve data from the file manager.



1 of 2



2 of 2

In the diagram above(that is broken into two parts), the relationships between objects in the ALMS is shown. The diagram 1 of 2 arrows point to Transaction. This displays the

inheritance that BorrowTransaction and ReturnTransactions contain. The Library manages Book and User Objects. Transactions operate on the Library. This is displayed with the dependency arrow for its process method. The FileManager saves and loads a Library for saving and retrieving information. On the right of 2 of 2 diagrams, the inheritance arrows show what each object points to. Book and User inherit BaseEntity and Searchable.

In conclusion, the ALMS implemented shows that we achieved modular design using encapsulation, inheritance, polymorphism, and adaptability.